

Laboratory work 7

Strings

Objective: study of basic algorithms for working with strings.

Short theoretical information

7.1. Concepts and announcements.

Unlike other languages, programming in C is not defined special type for string processing. Therefore, any a sequence of characters enclosed in double quotes "" is treated as string constant. Any line for correct output must end with a null character '\0' = 0. When declared string constant, the null character is added to it automatically. So, a sequence of characters representing a string constant, will be placed in the computer's RAM, including null byte.

That is, a string of characters is treated as an array of type elements char that ends with '\0', which is the end-of-line character. Then an array of characters (string) can be declared as follows:

```
char <string name> [ string length ]
```

Let's consider examples of declaration and initialization of strings:

```
const char p[]="We study programming";
```

```
char slovo[]="University";
```

```
char phrase1[11], phrase2[40];
```

In the described example, the constant p is declared, which is initialized with the value "We are learning programming", characters arrays slovo (without specifying the size), fraza1 (can contain up to 10 characters) and fraza2 (up to 39 characters).

The slovo character array can be declared in another way:

```
char slovo[11]= "University";
```

or

```
char slovo[]={ 'U', 'n', 'i', 'v', 'e', 'r', 's', 'i', 't', 'y', '\0' }
```

In the last example of string initialization slovo is necessary add a null character manually, otherwise the compiler will interpret it variable slovo not as a string, but as an array.

Memory cells are allocated to store the row, which go sequentially. For example, consider the representation of the variable pr of char type in RAM (Fig. 7.1), if it was assigned the value 'KI PROGRAMMER 11'.

▶	K	I		P	R	O	G	R	A	M	M	E	R		1	1
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Fig. 7.1

For placement in a line constant of some service symbols, character combinations are used. Yes, if it is necessary to include a double-quote symbol in the line, it must preceded by the "backslash" character: "\". String constants are placed in static memory. The starting address of a sequence of characters in double quotes

interpreted as a string address.

7.2. Operations on strings.

1. Assignment of lines.

One of the ways of assigning a string a certain value in the language C is the assignment of individual symbols, for example:

```
char str1[5];  
str1[0] = 'a';  
str1[1] = 'b';  
str1[2] = 'c';  
str1[3] = 'd';  
str1[4] = '\0';
```

However, this method is not convenient. A more convenient way assigning a string a certain value or copying the value of one string to another is to use the `strcpy()` function:

```
char *strcpy (char *destination, const char *source),
```

where `destination` is a pointer to the line to be copied to result, `source` – a pointer to a line, the source of copying (what will be copied), for example:

```
char str1[10], str2[10];  
strcpy(str1, "Hello");  
strcpy(str2, str1);
```

In the given example, the character string `str1` is assigned the value of `Hello`. Next, the value of the character string `str1` is copied to the character string `str2`, that is, `str2` also contains the value `Hello`.

However, to avoid overflow, it is necessary to control the length of the string into which another string is copied (including the null character).

In the C programming language, to work with all string processing functions, it is necessary to connect the `string.h` library.

It should be noted that the C language does not allow the use of the construction of the assignment of a string value of the type:

```
char str1[10], str2[10];  
str1="Hello"; // Error  
str2=str1; // Error
```

As a result of executing this code, the compiler will issue error message (not to be confused with string initialization: `char str[10] = "Hello";` such code will be correct).

2. Combining strings (concatenation) allows you to add one line to the end of another line, the operation is marked with a “+” sign. During string concatenation, the current length of the concatenated string is increased by the length of the concatenated string, and if there is not enough space in the receiving string, the extra characters are discarded.

You can combine strings using the function:

```
char *strcat (char *destination, const char *append),
```

where `destination` is a pointer to the array to which the line will be added, `append` is a pointer to the array from which the line will be copied.

This function adds all characters of line 2 to line 1.

The resulting string is terminated by a null character.

3. Comparison of lines. Character strings can be compared between by myself Two strings are compared from left to right character-wise, with 'A' < 'B'. The character that is placed further in the alphabet (it has a higher number in the table of ASCII codes) is considered larger. The strcmp() function is used to compare strings:

```
int strcmp (const char *str1, const char *str2);
```

where str1 and str2 are pointers to strings to be compared, and string comparison is case-sensitive.

The result of this function is a negative number (if line1 is less than line2), 0 (if the lines are the same) or a positive number (if line1 is greater than line2), For example:

```
int n = strcmp("Program", "program");  
printf("%d", n);
```

The result of the execution will be -1, that is, row1 is less than line2 (Program<program). And indeed, since the line comparison is character-by-character, and in the ASCII table 'P'=80 and 'p'=112, and the rest of the characters are the same, the result will be Program<program.

7.3. Strings input–output.

To input and output lines to the console in the C language the gets() and puts() functions are used, respectively.

The gets() function reads a string of characters entered from the keyboard, and places them at the address specified in the argument. You can type characters until the enter key is pressed. The character corresponding to the enter key - the carriage return - will not be part of the string. Instead, a null character will appear at the end of the string, and gets() will terminate. It's actually not possible to use gets() to get a carriage return (you can use getchar() and its variants). If mistakes were made during entry, they can be corrected by pressing the BACKSPACE key before by pressing the enter key.

The puts() function displays the argument passed to it on the screen, ending the output with a new line in the console.

For example, the following code snippet reads the string from keyboard and counts the number of characters.

```
char str[80];  
gets(str);  
printf("Length is %d", strlen(str));
```

Below are the main functions of the console input/output:

getchar() – reads a character from the keyboard, waits for a keystroke input;

getche() – reads a character from the keyboard and outputs it to screen;

getch() – reads a character from the keyboard, but does not output it on the screen;

putchar() – displays a character on the screen;

gets() – reads a string from the keyboard;

puts() – displays a string on the screen.

7.4. Other functions for processing strings.

A function is used to determine the length of a string strlen():

size_t *strlen (const char *str),

where str is a pointer to a line - determines the actual number characters in a string. It accepts one parameter - a string of characters, and returns an integer as a result of its work - the length of this string.

The function of splitting a line into separate sublines as specified strtok() delimiter:

char *strtok(char *str, const char *sep),

where str is a pointer to the input string, sep is a separator string.

The strtok() function selects the next part of the string it points to the str argument, separated by one of the delimiter characters specified in the string pointed to by the sep argument. Successive calls to the strtok() function result in splitting the string str into parts (tokens).

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <locale.h>
```

```
int main(void)
```

```
{
```

```
    setlocale(LC_ALL, "ukr");
```

```
    // Array with string to search
```

```
    char str[40] = "test1 / test2 / test3 / test4";
```

```
    // A set of characters that should be included in the searched segment
```

```
    char sep[4] = " / ";
```

```
    // A variable in which the initial addresses of the parts will be entered line str
    char *istr;
```

```
    printf("Starting line: %s\n", str);
```

```
    printf("Splitting result : \n");
```

```
    // Selection of the first part of the line
```

```
    istr = strtok(str, sep);
```

```
    // Selection of the following parts
```

```
    while (istr != NULL)
```

```
    {
```

```
        //output of the next selected part
```

```
        printf("%s\n", istr);
```

```
        // selection of the next part of the line
```

```
        istr = strtok(NULL, sep);
```

```
    }
```

```
    return 0;
```

```
}
```

Performance result

Initial string: "test1 / test2 / test3 / test4"

Breakdown result

test1

test2

test3

test4

The strchr() function of the first occurrence of a string:

char *strchr (const char *str, int ch),

where str is a pointer to the line in which the search will be performed, ch is the code of the searched character.

This function returns a pointer to the first occurrence of the ch symbol in the string str; if not found, returns NULL.

```
#define _CRT_SECURE_NO_WARNINGS
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <locale.h>
```

```
int main(void)
```

```
{
```

```
    setlocale(LC_ALL, "ukr");
```

```
    char str[11] = "0123456789";
```

```
    // The code of the searched character
```

```
    int ch = '6';
```

```
    // Pointer to the searched variable in the string being executed  
    search
```

```
    char *ach;
```

```
    // Looking for character '6'
```

```
    ach = strchr(str, ch);
```

```
    if (ach == NULL)
```

```
        printf("Character not found\n");
```

```
    else
```

```
        printf("Searched character in line at position # %d\n",
```

```
        ach - str + 1);
```

```
    return 0;
```

```
}
```

Performance result

Searched character in line at position #7

The function of finding the first occurrence of string_A in string_B strstr():

char *strstr(const char *string_B, const char *string_A).

The strstr() function searches for the first occurrence of a string (for except for the end-of-line character) pointed to by the line_A argument to the line pointed to by the line_B argument. If string_A has zero length, the function will return a pointer to the beginning of the string string_B.

```
#define _CRT_SECURE_NO_WARNINGS
```

```

#include <stdio.h>
#include<string.h>
#include <locale.h>
int main(void)
{
    setlocale(LC_ALL, "ukr");
    char str1[11] = "0123456789";
    // A set of characters that should be included in the searched sequence
    char str2[3] = "67";
    // A variable into which the address of the first line found will be placed
    char *istr;
    //Search for a string
    istr = strstr(str1, str2);
    if (istr == NULL)
        printf("String not found\n");
    else
        printf("Searched string starts with character %d\n", istr - str1 + 1);
    return 0;
}

```

Performance result

The searched string starts with the character 7

Example 7.1.

The specified text, the words of which are separated by a space, comma, or period. Highlight and print all words of the text, each word on a new line.

To solve the problem, you need:

- 1) declare two string variables, one for the input text, the second for the highlighted word;
- 2) declare three integer variables for finding the positions of the space, comma, and period, respectively, and one integer variable for the position of the first punctuation mark;
- 3) read the input line;
- 4) determine the positions of separator characters in the input text;
- 5) determine the position of the delimiter symbol that occurs first by comparing the non-zero values of the index variables;
- 6) select the word from the first position to the delimiter from the input line and print it;
- 7) delete the part from the first character to the separator character from the input line;
- 8) if the position of the first delimiter character is greater than zero, go to point 4;
- 9) output the rest of the input text as one word, since in it has no separators.

Program text

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include<string.h>
#include<locale.h>
int main(void)
{
    setlocale(LC_ALL, "ukr");
    char str[80];
    char * t;
    printf("\n");
    printf("\nEnter the string:\n");
    get(str);
    t = strtok(str, " ,.");
    printf("\n");
    setlocale(LC_ALL, "C");
    while (t != NULL) {
        printf("\n%s", t);
        t = strtok(NULL, " ,.");
    }
    printf("\n");
    return 0;
}
```

The result of the program

```
Enter the line:
Hello, my name is Vasyl
We will get:
Hello
me
call
Vasyl
```

Example 6.2.

Find all the words in the given line and enclose them in quotation marks. Print the modified string.

To solve the problem, it is necessary to perform character-by-character copying of one line into another with additional control of changing letters to symbols and vice versa. With each change, you must add a quotation mark to the recipient string. The start and end of the line must be controlled.

Program text

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include<string.h>
int main(void)
{
```

```

char s[50], t[50], w[] = "";
int i, j = 0;
printf("Enter row\n");
get(s);
i = 0;
while (i <= strlen(s))
{
while ((s[i]<'A' || s[i]>'Z') && (s[i]<'a' ||
s[i]>'z'))// until letter
t[j++] = s[i++];//add the i-th character to the line
if (i <= strlen(s))
t[j++] = "\n";
else
break
while ((s[i] >= 'a' && s[i] <= 'z') || (s[i] >= 'A'
&& s[i] <= 'Z'))// if the i-symbol is a letter, add it to the output
line
t[j++] = s[i++];
t[j++] = "\n";// add quotes
}
for (i = 0; i < strlen(t); i++)
{
printf("%c", t[i]);
}
return 0;
}

```

The result of the program

Input string 123//rett 44tyr0 ---tree stop

123// "rett" 44 "tyr" 0 --- "tree" "stop"

Tasks of laboratory work

Task 1

1. Delete the symbols "+", "-", "*" from the given line, and place to put a space.
2. Find all lines of "ABC" in the text, replace them with lowercase letters.
3. Two lines are given. Print characters that match and are in the same positions.
4. A string containing upper and lower case letters is specified. Print uppercase letters of the string first, then lowercase. The rest of the characters are not on the screen withdraw.

Task 2

1. Given a text whose words are separated by a colon, a semicolon, a hyphen, and a space. Highlight, print and count the number of words. Display the results on the screen.
2. Given a string, find all words enclosed in quotation marks and output each word without quotation marks one per line.

3. In the given line, find all the words that start with or uppercase letters, or end with a capital letter and replace uppercase letters with lowercase letters.
4. In the given line, the words are separated by one or more spaces. Remove all extra spaces from the line so that there is one space between each word.

Task 3.

1. Find the shortest and longest word in the given line, output these words and their length.
2. Find all numbers that read the same in the given line on both sides, and display on the screen, if not display message.
3. In the specified line, the numbers are separated by “+” and “*” signs. Calculate the value of the expression.
4. You need to create a string consisting of substrings that are entered from the keyboard in the following format: serial_number: substring; interline_number: subline; When forming sequence numbers are not included on a new line, but sublines must be placed according to their sequence numbers.